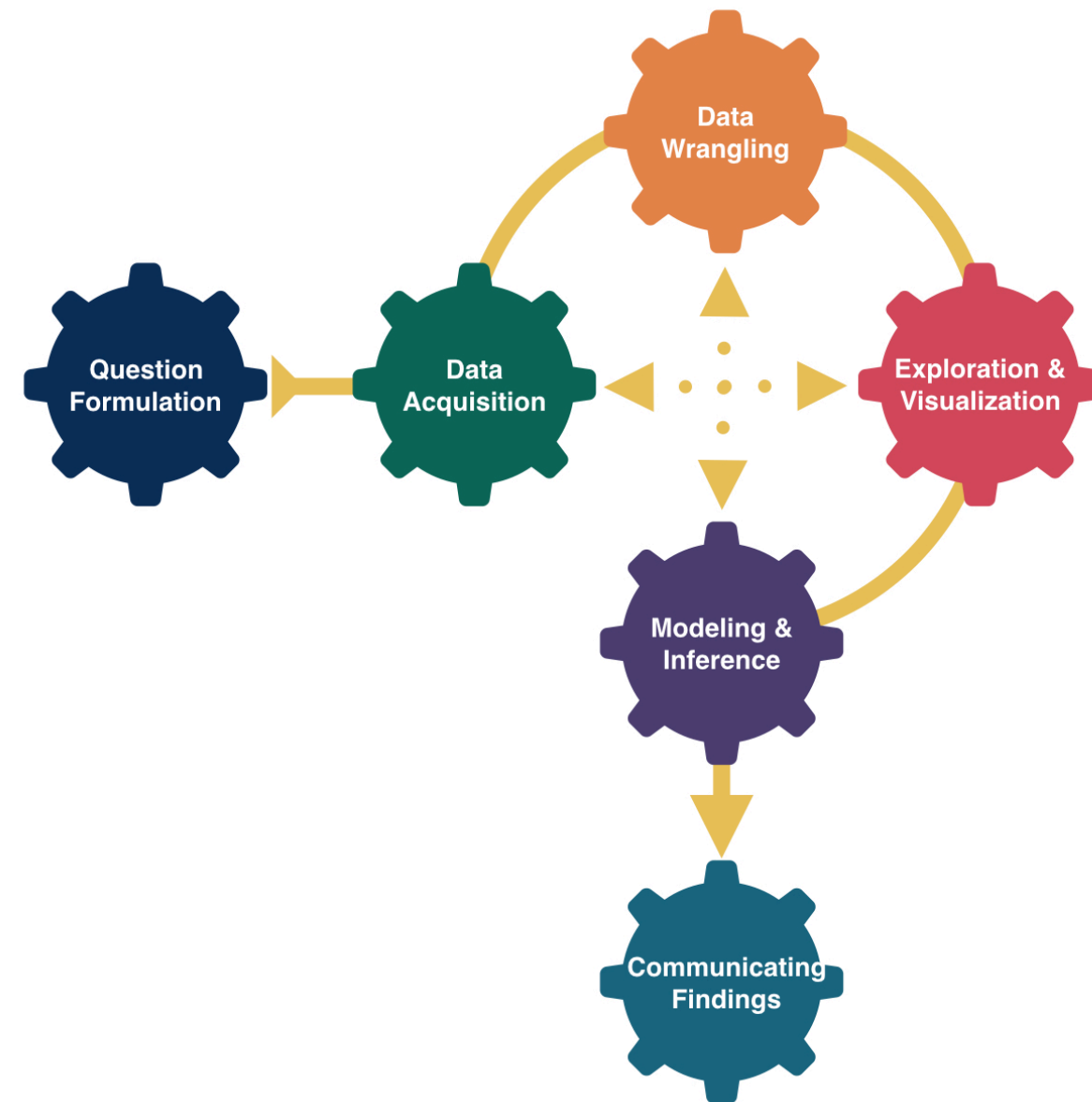


Data Wrangling & Summarization



Kelly McConville

DATA 100

Week 3 | Spring 2026

Announcements

- Code a Valentine: Feb 10th, Taylor 212

Goals for Today

- Consider measures for **summarizing** quantitative data
 - Center
 - Spread/variability
- Consider measures for **summarizing** categorical data
- Define **data wrangling**
- Learn to use functions in the **dplyr** package to summarize and wrangle data

Load Necessary Packages



dplyr is part of the **tidyverse** collection of data science packages.

```
1 # Load necessary packages
2 library(tidyverse)
```

Import the Data

```
1 july_2019 <- read_csv("data/july_2019.csv")
2
3 # Inspect the data
4 glimpse(july_2019)
```

Rows: 192

Columns: 8

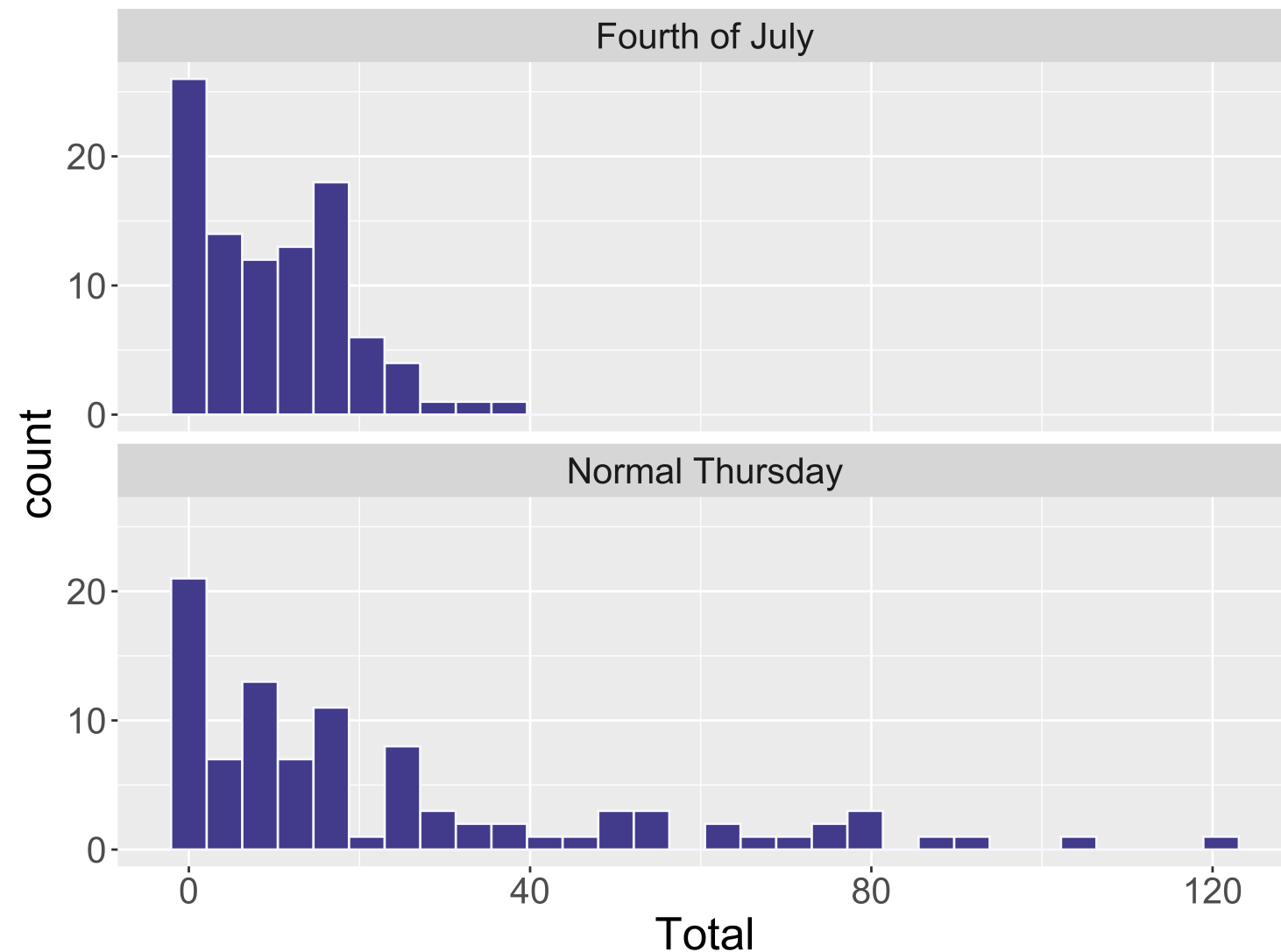
```
$ DateTime <chr> "07/04/2019 12:00:00 AM", "07/04/2019 12:15:00 AM", "07/04/2...
$ Day      <chr> "Thursday", "Thursday", "Thursday", "Thursday", "Thursday", ...
$ Date     <date> 2019-07-04, 2019-07-04, 2019-07-04, 2019-07-04, 2019-07-04,...
$ Time     <time> 00:00:00, 00:15:00, 00:30:00, 00:45:00, 01:00:00, 01:15:00,...
$ Total    <dbl> 2, 3, 2, 0, 3, 2, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, ...
$ Westbound <dbl> 2, 3, 1, 0, 2, 2, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, ...
$ Eastbound <dbl> 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, ...
$ Occasion <chr> "Fourth of July", "Fourth of July", "Fourth of July", "Fourt...
```

Summarizing Data

Hard to do by eyeballing a spreadsheet with many rows!

DateTime	Day	Date	Time	Total	Westbound	Eastbound	Occasion
07/04/2019 06:00:00 AM	Thursday	2019- 07-04	06:00:00	1	1	0	Fourth of July
07/04/2019 06:15:00 AM	Thursday	2019- 07-04	06:15:00	4	0	4	Fourth of July
07/04/2019 06:30:00 AM	Thursday	2019- 07-04	06:30:00	9	1	8	Fourth of July
07/04/2019 06:45:00 AM	Thursday	2019- 07-04	06:45:00	5	0	5	Fourth of July

Summarizing Data Visually



For a quantitative variable, want to answer:

- What is an **average** value?
- What is the **trend/shape** of the variable?
- How much **variation** is there from case to case?

Need to learn key **summary statistics**: Numerical values computed based on the observed cases.

Measures of Center

Mean: Average of all the observations

- n = Number of cases (sample size)
- x_i = value of the i -th observation
- Denote by $\bar{x}$

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

```
1 # Test out on first 6 values
2 head(july_2019$Total)
```

```
[1] 2 3 2 0 3 2
```

Compute with a **dplyr** function:

```
1 summarize(july_2019, mean_bikes = mean(Total))
```

```
# A tibble: 1 × 1
```

```
mean_bikes
```

```
<dbl>
```

```
1      17.1
```

Measures of Center

Median: Middle value

- Half of the data falls below the median
- Denote by m
- If n is even, then it is the average of the middle two values

```
1 # Test out on first 6 values
2 head(july_2019$Total)
```

```
[1] 2 3 2 0 3 2
```

Compute with a **dplyr** function:

```
1 summarize(july_2019, median_bikes = median(Total))
```

```
# A tibble: 1 × 1
```

```
  median_bikes
```

```
    <dbl>
```

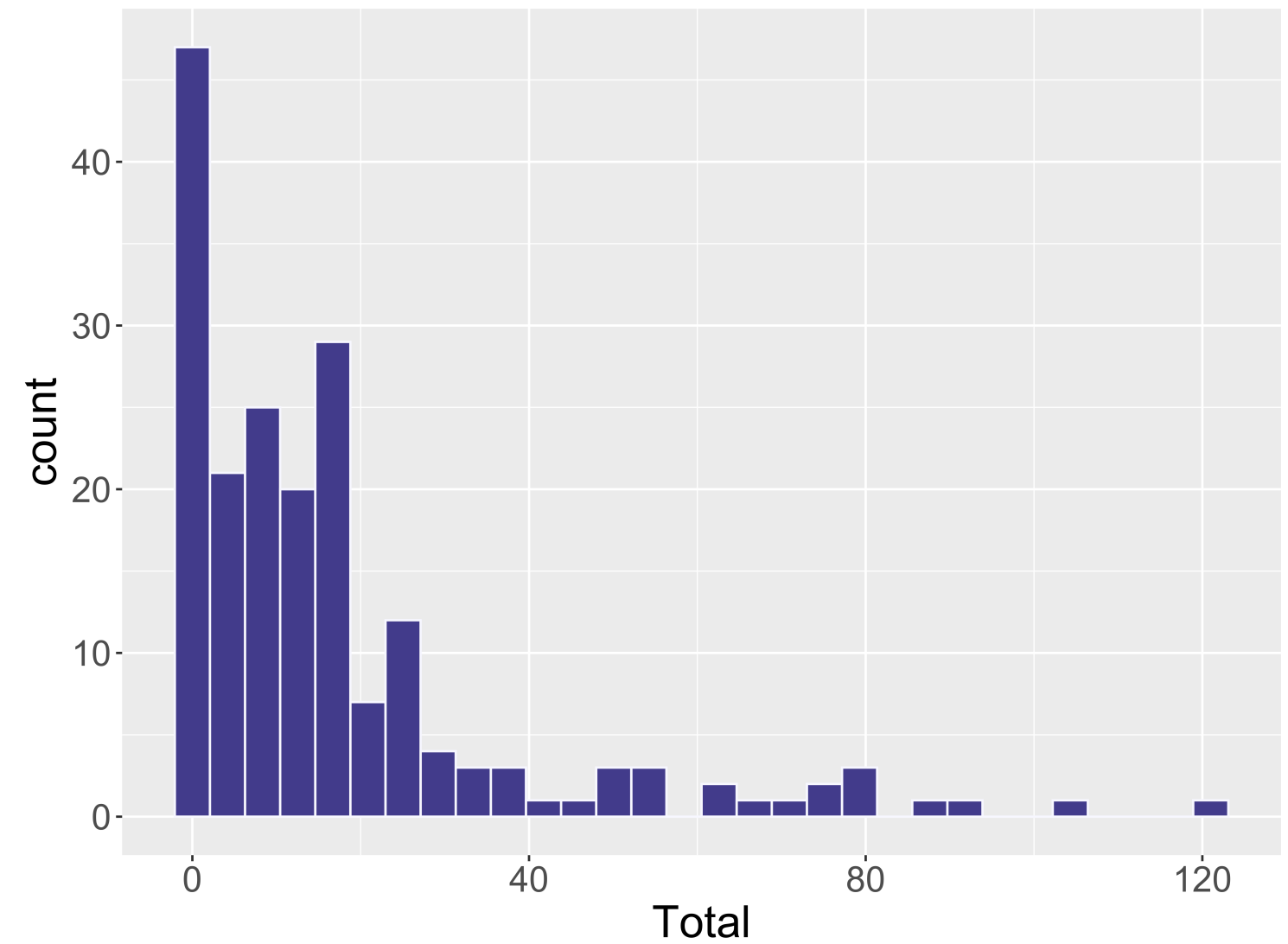
```
1         11
```

Measures of Center

Why is the mean larger than the median?

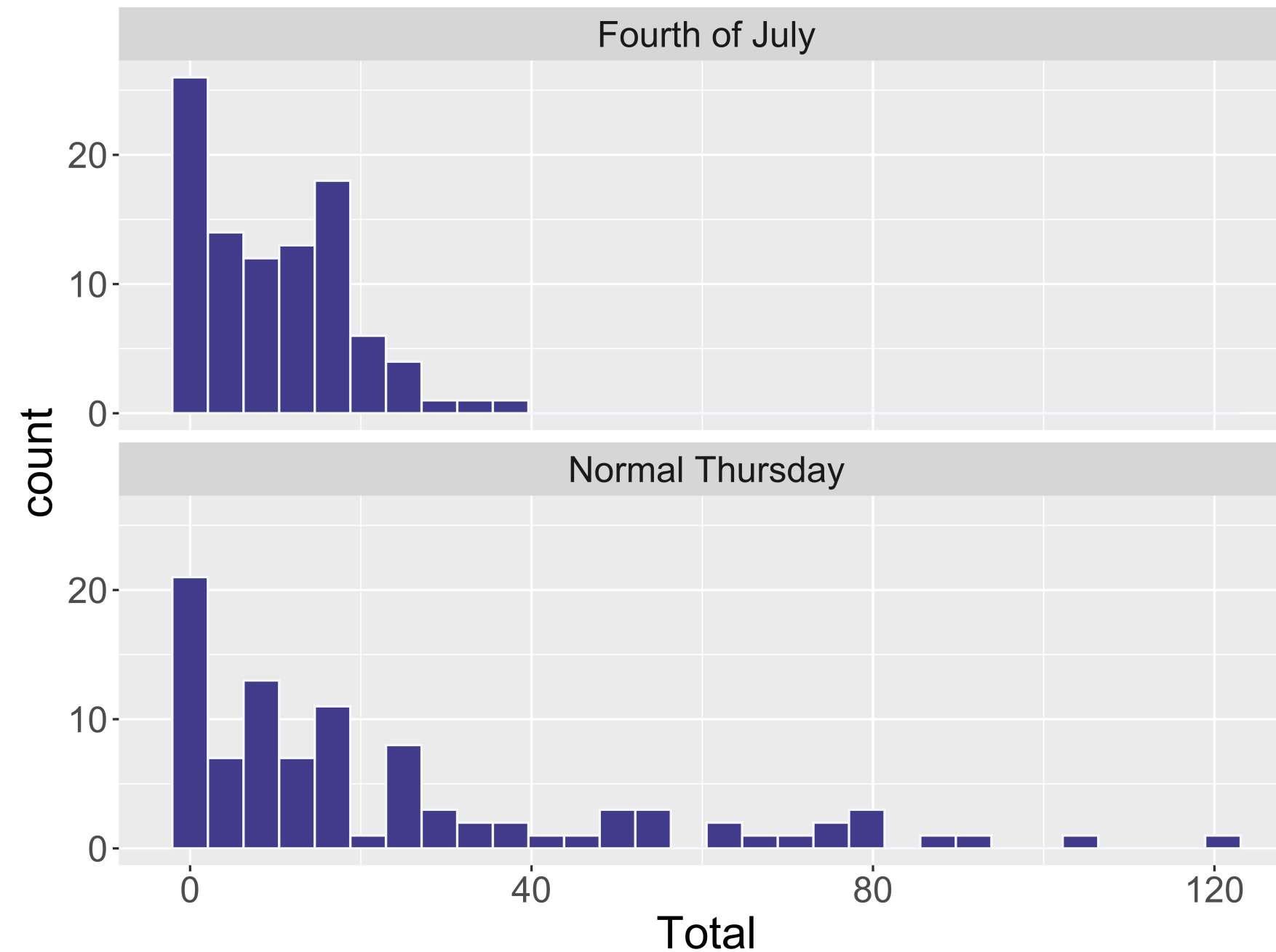
```
1 summarize(july_2019, mean_bikes = mean(Total),  
2           median_bikes = median(Total))
```

```
# A tibble: 1 × 2  
  mean_bikes median_bikes  
    <dbl>      <dbl>  
1     17.1         11
```



Computing Measures of Center by Groups

Question: Were there more bikes, on average, for Fourth of July or for the normal Thursday?



Computing Measures of Center by Groups

Handy `dplyr` function: `group_by()`

```
1 july_2019_grouped <- group_by(july_2019, Occasion)
2 july_2019_grouped
```

```
# A tibble: 192 × 8
```

```
# Groups:   Occasion [2]
```

	DateTime		Day	Date	Time	Total	Westbound	Eastbound	Occasion
	<chr>		<chr>	<date>	<tim>	<dbl>	<dbl>	<dbl>	<chr>
1	07/04/2019 12:00:0...		Thur...	2019-07-04	00:00	2	2	0	Fourth ...
2	07/04/2019 12:15:0...		Thur...	2019-07-04	00:15	3	3	0	Fourth ...
3	07/04/2019 12:30:0...		Thur...	2019-07-04	00:30	2	1	1	Fourth ...
4	07/04/2019 12:45:0...		Thur...	2019-07-04	00:45	0	0	0	Fourth ...
5	07/04/2019 01:00:0...		Thur...	2019-07-04	01:00	3	2	1	Fourth ...
6	07/04/2019 01:15:0...		Thur...	2019-07-04	01:15	2	2	0	Fourth ...
7	07/04/2019 01:30:0...		Thur...	2019-07-04	01:30	1	1	0	Fourth ...
8	07/04/2019 01:45:0...		Thur...	2019-07-04	01:45	0	0	0	Fourth ...
9	07/04/2019 02:00:0...		Thur...	2019-07-04	02:00	0	0	0	Fourth ...
10	07/04/2019 02:15:0...		Thur...	2019-07-04	02:15	0	0	0	Fourth ...

```
# i 182 more rows
```

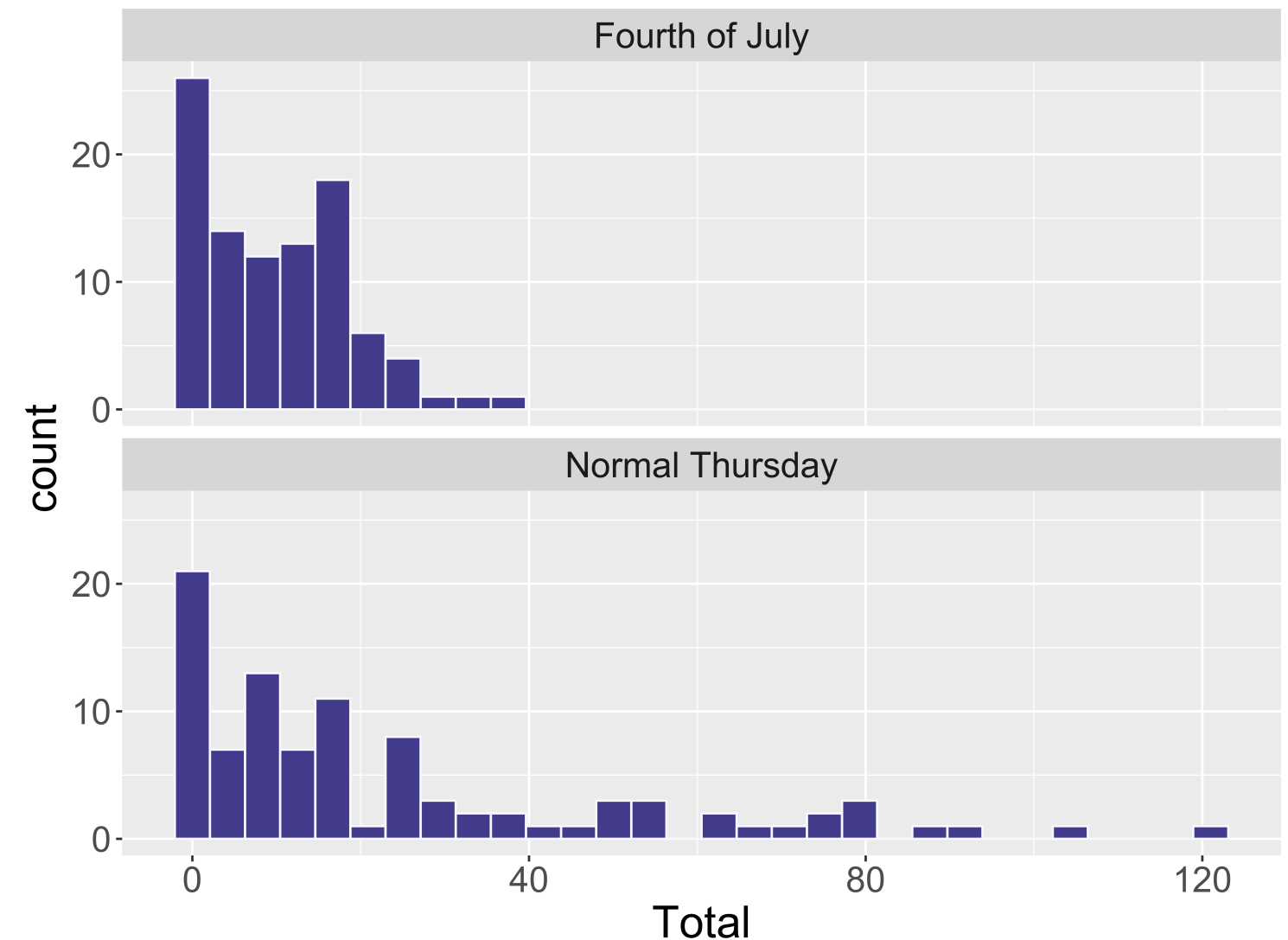
Computing Measures of Center by Groups

Compute summary statistics on the grouped data frame:

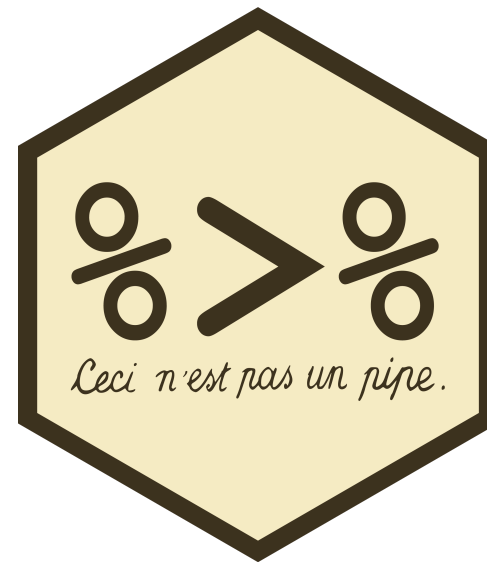
```
1 july_2019_grouped <- group_by(july_2019, Occasion)
2 summarize(july_2019_grouped,
3           mean_bikes = mean(Total),
4           median_bikes = median(Total))
```

A tibble: 2 × 3

	Occasion	mean_bikes	median_bikes
	<chr>	<dbl>	<dbl>
1	Fourth of July	10.0	9
2	Normal Thursday	24.2	14.5



And now it is time to learn the pipe: %>%



Chaining **dplyr** Operations

Instead of:

```
1 july_2019_grouped <- group_by(july_2019, Occasion)
2 summarize(july_2019_grouped,
3           mean_bikes = mean(Total),
4           median_bikes = median(Total))
```

```
# A tibble: 2 × 3
  Occasion      mean_bikes median_bikes
  <chr>          <dbl>         <dbl>
1 Fourth of July      10.0             9
2 Normal Thursday     24.2            14.5
```

Use the pipe:

```
1 july_2019 %>%
2   group_by(Occasion) %>%
3   summarize(mean_bikes = mean(Total),
4             median_bikes = median(Total))
```

```
# A tibble: 2 × 3
  Occasion      mean_bikes median_bikes
  <chr>          <dbl>         <dbl>
1 Fourth of July      10.0             9
2 Normal Thursday     24.2            14.5
```

- Why pipe?
- You can also use `|>`, which is newer and often referred to as the “base **R** pipe.”

Measures of Variability

- Want a statistic that captures how much observations **deviate** from the mean
- Find how much each observation deviates from the mean.
- Compute the average of the deviations.

```
1 # Test out on first 6 values
2 head(july_2019$Total)
[1] 2 3 2 0 3 2
```

$$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})$$

Problem?

Measures of Variability

- Want a statistic that captures how much observations **deviate** from the mean

Here is my **NEW** proposal:

- Find how much each observation deviates from the mean.
- Compute the average of the **squared** deviations.

```
1 # Test out on first 6 values  
2 head(july_2019$Total)
```

```
[1] 2 3 2 0 3 2
```

Measures of Variability

- Want a statistic that captures how much observations **deviate** from the mean

Here is my **ACTUAL** formula:

- Find how much each observation deviates from the mean.
- Compute the (nearly) average of the **squared** deviations.
- Called **sample variance** s^2 .

$$s^2 = \frac{1}{n - 1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Compute with a **dplyr** function:

```
1 summarize(july_2019, var_bikes = var(Total))  
# A tibble: 1 × 1  
  var_bikes  
    <dbl>  
1      454.
```

Measures of Variability

- Want a statistic that captures how much observations **deviate** from the mean
- Find how much each observation deviates from the mean.
- Compute the (nearly) average of the **squared** deviations.
- Called **sample variance** s^2 .
- The square root of the sample variance is called the **sample standard deviation** s .

Compute with a **dplyr** function:

```
1 summarize(july_2019, var_bikes = var(Total),  
2           sd_bikes = sd(Total))  
  
# A tibble: 1 × 2  
  var_bikes sd_bikes  
    <dbl>    <dbl>  
1     454.     21.3
```

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Measures of Variability

- In addition to the sample standard deviation and the sample variance, there is the sample **interquartile range** (IQR):

$$\text{IQR} = Q_3 - Q_1$$

Compute with a **dplyr** function:

```
1 summarize(july_2019, iqr_bikes = IQR(Total))  
# A tibble: 1 × 1  
  iqr_bikes  
    <dbl>  
1         16
```

Comparing Measures of Variability

- Which is more robust to outliers, the IQR or s ?
- Which is more commonly used, the IQR or s ?

```
1 july_2019 %>%
2   group_by(Occasion) %>%
3   summarize(sd_bikes = sd(Total),
4             iqr_bikes = IQR(Total))
```

A tibble: 2 × 3

	Occasion <chr>	sd_bikes <dbl>	iqr_bikes <dbl>
1	Fourth of July	8.30	14
2	Normal Thursday	27.2	27.2

Summarizing Categorical Variables

Return to the Buffalo Bills 2018 Season Data

Focus on the dogs with the 5 most common names

```
1 # Pull in the dataset on all teams
2 nfl_outcomes_2018 <- read_csv("https://data.scorenetwork.org/data/nfl_mahomes_era_games.csv")
3
4 # Focus on just the Bills
5 buf <- filter(nfl_outcomes_2018, home_team == "BUF" | away_team == "BUF") %>%
6   mutate(where = if_else(home_team == "BUF", "home", "away"))
```

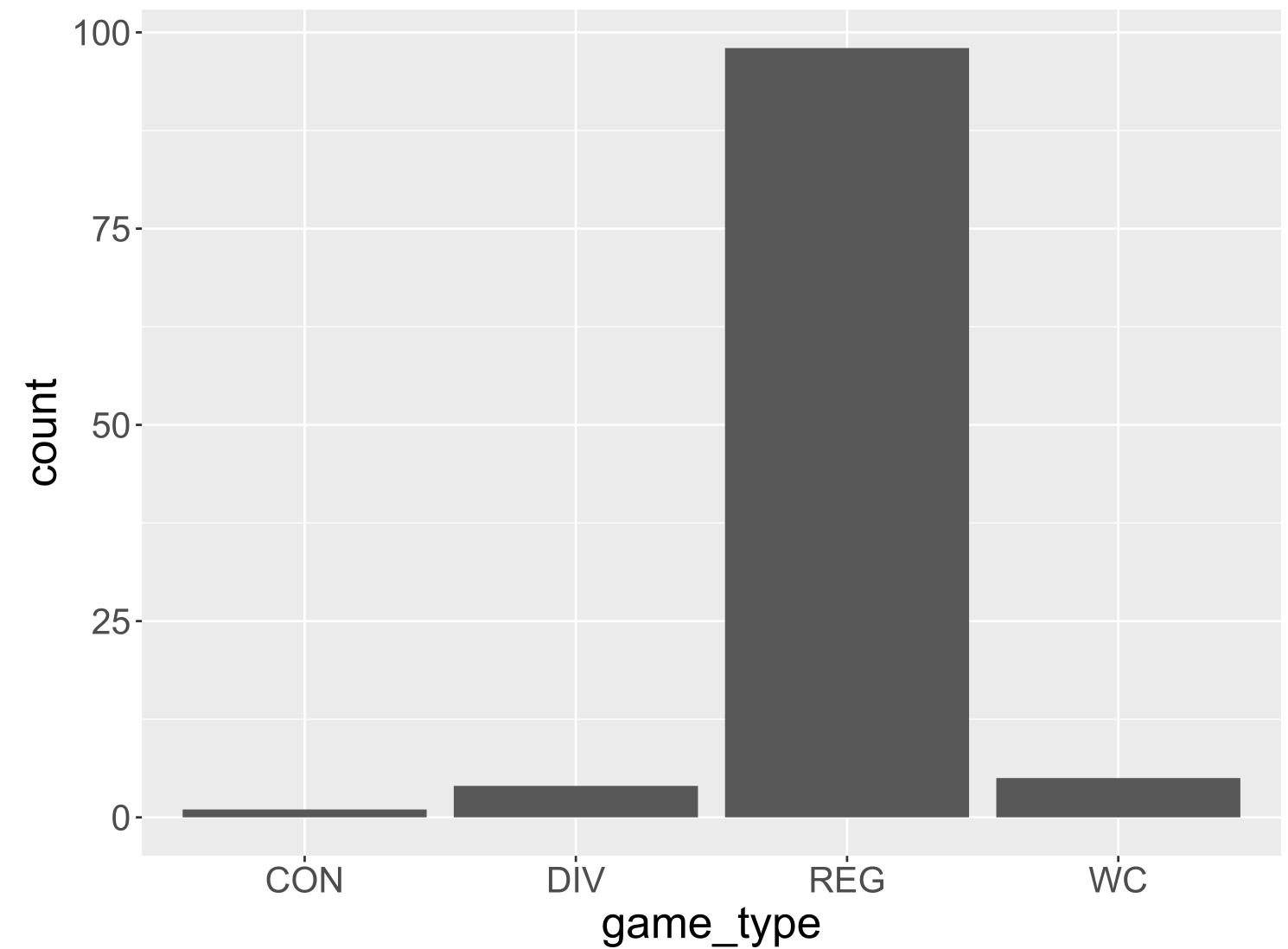
Frequency Table

```
1 count(buf, game_type)
```

```
# A tibble: 4 × 2
```

	game_type	n
	<chr>	<int>
1	CON	1
2	DIV	4
3	REG	98
4	WC	5

```
1 ggplot(data = buf,  
2         mapping = aes(x = game_type)) +  
3   geom_bar()
```



Frequency Table

```
1 count(buf, game_type)
```

```
# A tibble: 4 × 2
  game_type      n
  <chr>    <int>
1 CON         1
2 DIV         4
3 REG        98
4 WC          5
```

```
1 count(buf, game_type, sort = TRUE)
```

```
# A tibble: 4 × 2
  game_type      n
  <chr>    <int>
1 REG        98
2 WC          5
3 DIV         4
4 CON         1
```

geom_col() versus geom_bar()

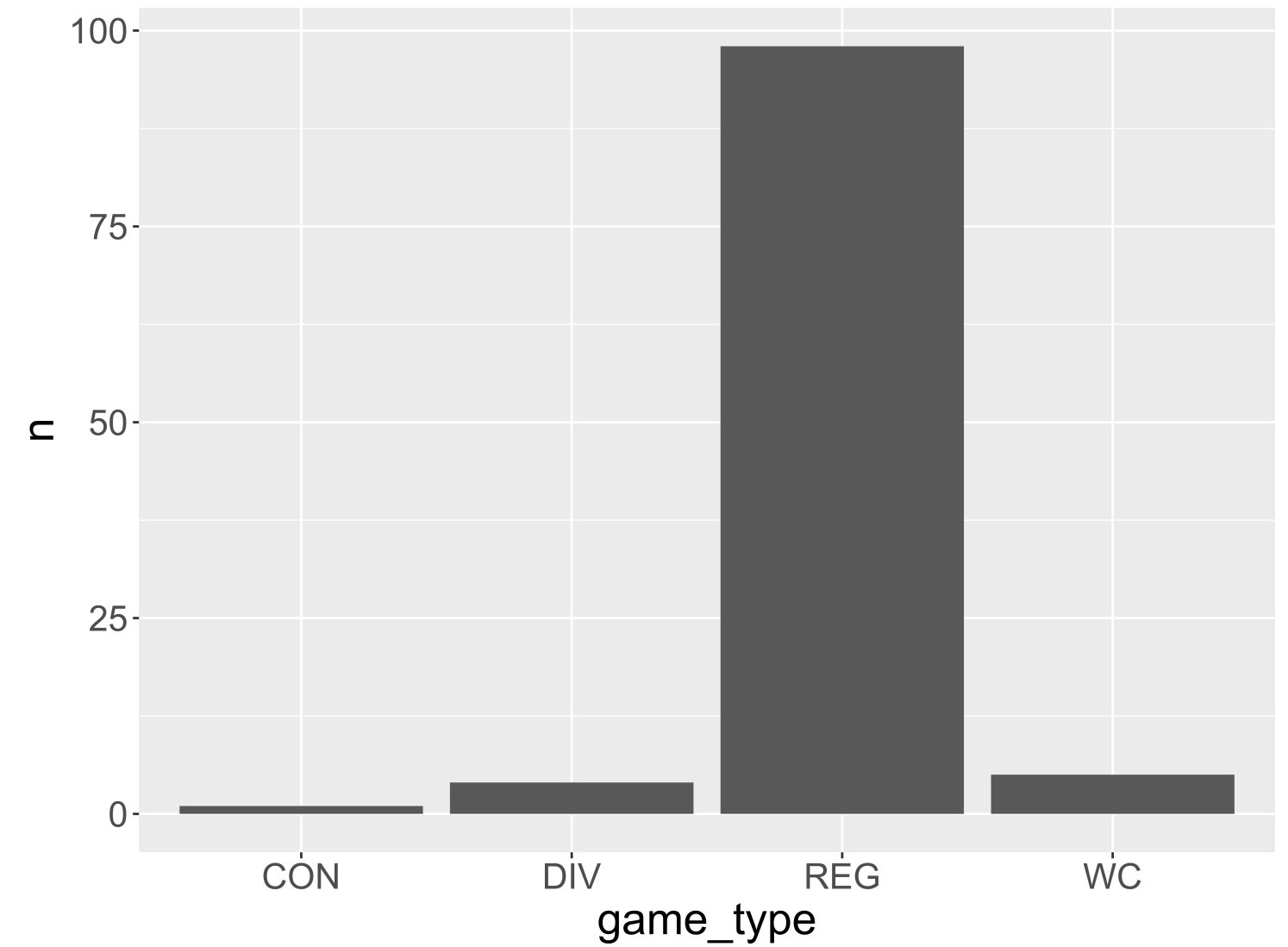
If you have already aggregated the data, you will use `geom_col()` instead of `geom_bar()`.

```
1 game_counts <- count(buf, game_type)
2 game_counts
```

```
# A tibble: 4 × 2
  game_type      n
  <chr>    <int>
1 CON         1
2 DIV         4
3 REG        98
4 WC          5
```

geom_col() Example

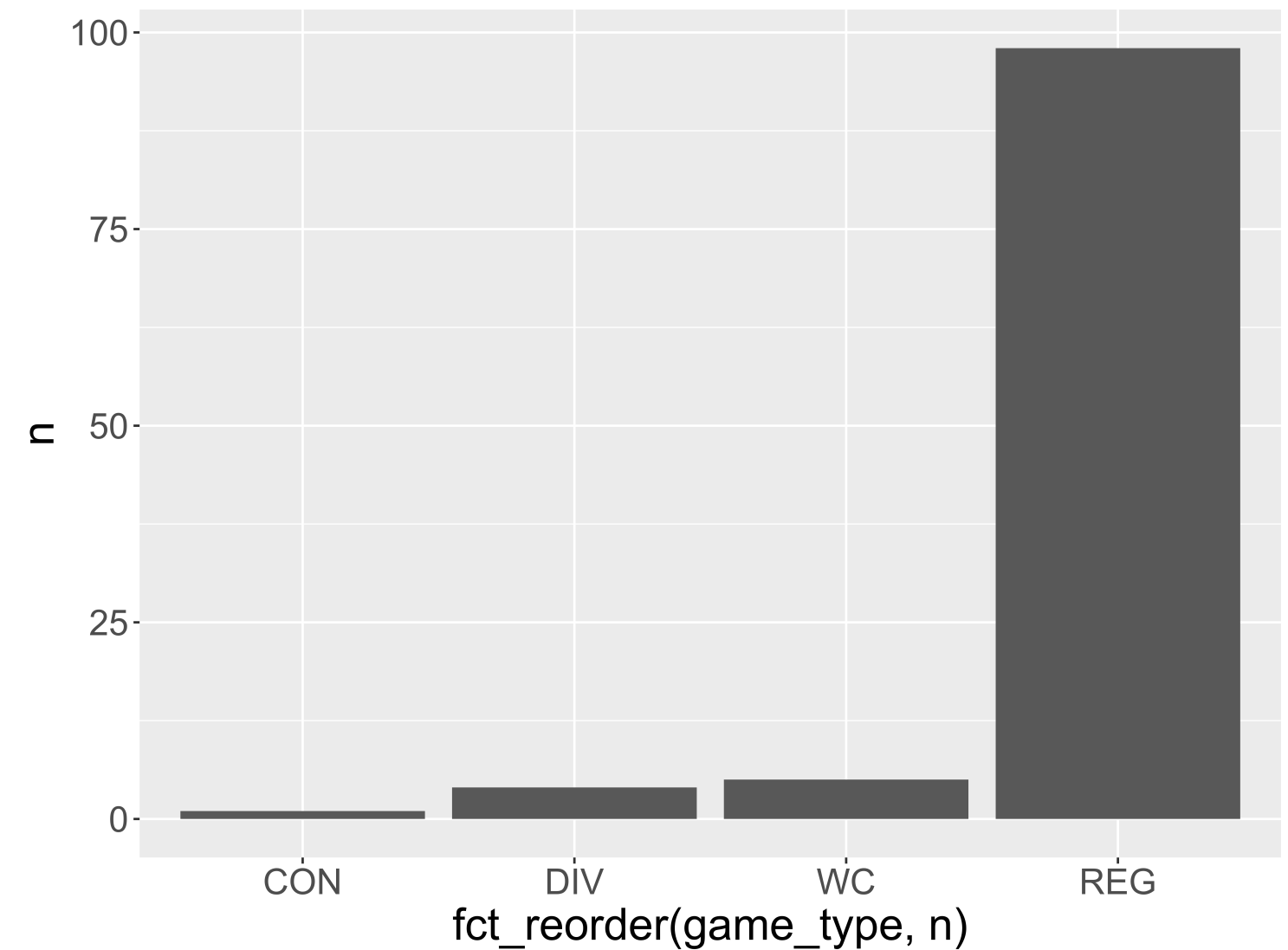
```
1 ggplot(data = game_counts,  
2       mapping = aes(x = game_type,  
3                     y = n)) +  
4   geom_col()
```



Another ggplot2 geom: geom_col()

And use `fct_reorder()` instead of `fct_infreq()` to reorder bars.

```
1 ggplot(data = game_counts,  
2       mapping = aes(x = fct_reorder(game_type, n),  
3                          y = n)) +  
4   geom_col()
```



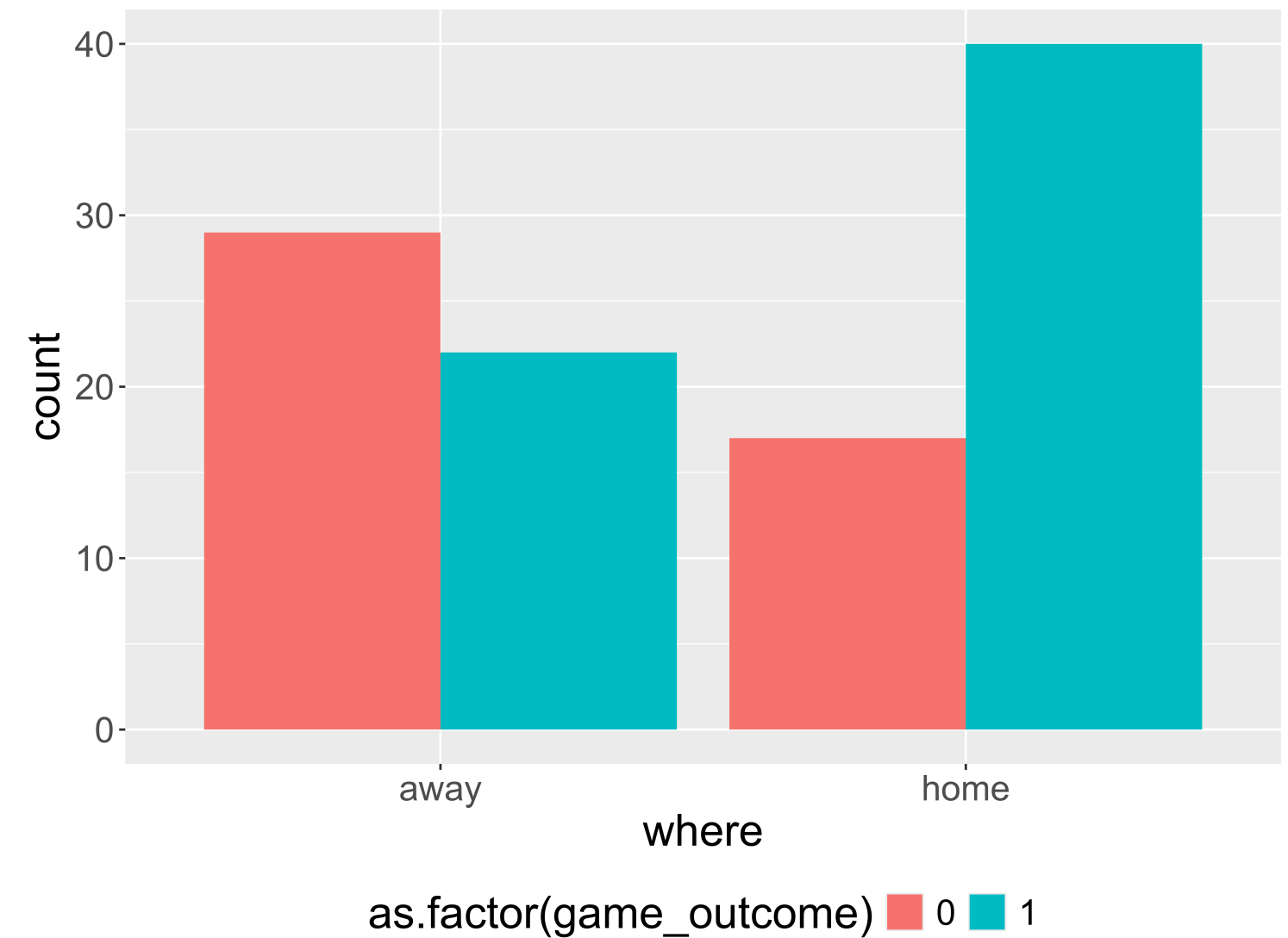
Contingency Table

```
1 count(buf, where, game_outcome)
```

A tibble: 4 × 3

	where	game_outcome	n
	<chr>	<dbl>	<int>
1	away	0	29
2	away	1	22
3	home	0	17
4	home	1	40

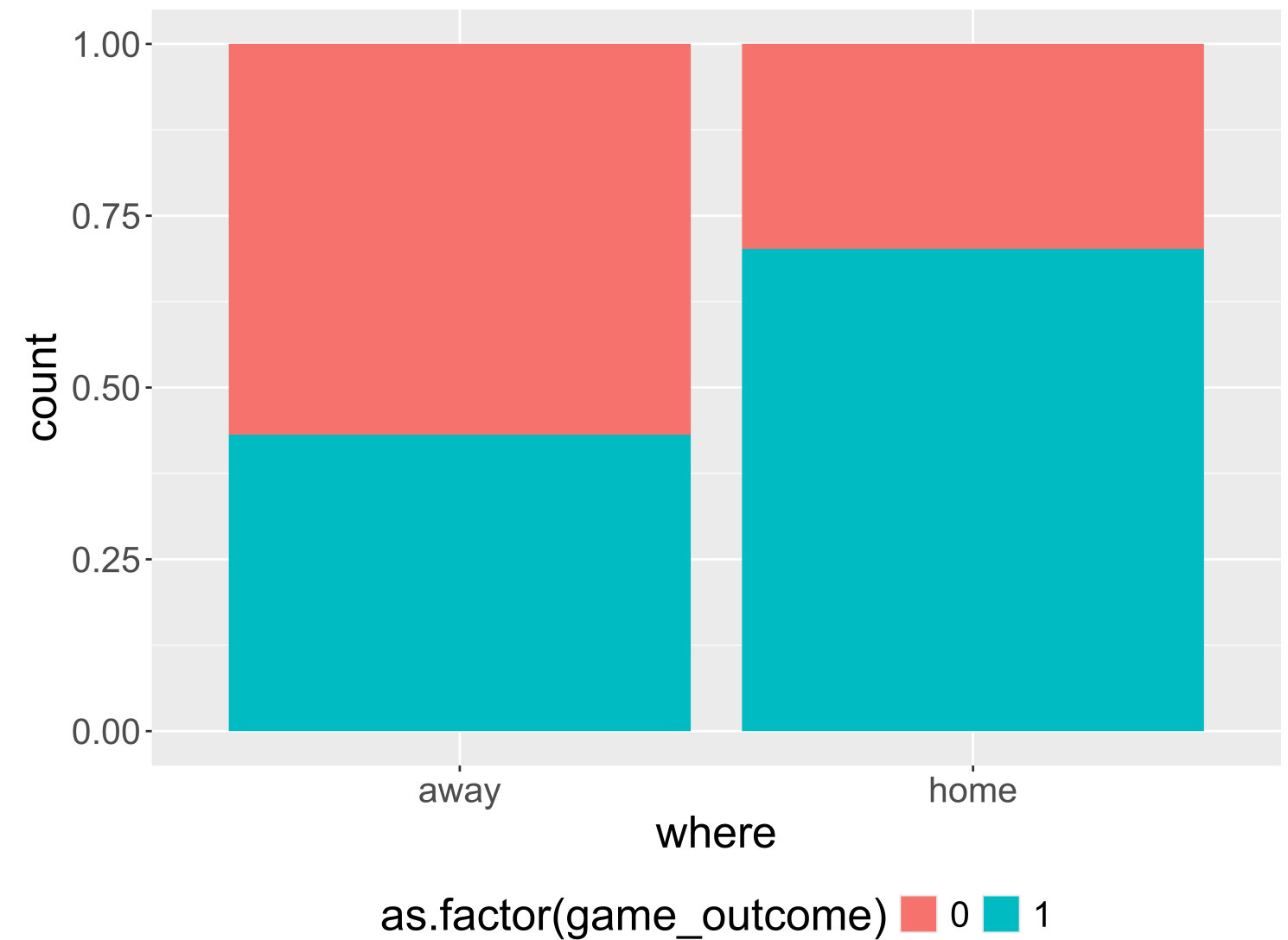
```
1 ggplot(data = buf,  
2       mapping = aes(x = where,  
3                     fill = as.factor(game_outcome))) +  
4   geom_bar(position = "dodge") +  
5   theme(legend.position = "bottom")
```



Conditional Proportions

- Beyond raw counts, we often summarize categorical data with **conditional** proportions.
 - Especially when looking for relationships!

```
1 ggplot(data = buf,  
2       mapping = aes(x = where,  
3                     fill = as.factor(game_outcome))) +  
4   geom_bar(position = "fill") +  
5   theme(legend.position = "bottom")
```



Conditional Proportions

```
1 count(buf, where, game_outcome)
```

```
# A tibble: 4 × 3
```

	where	game_outcome	n
	<chr>	<dbl>	<int>
1	away	0	29
2	away	1	22
3	home	0	17
4	home	1	40

```
1 count(buf, where, game_outcome) %>%
```

```
2   group_by(where) %>%
```

```
3   mutate(prop = n/sum(n))
```

```
# A tibble: 4 × 4
```

```
# Groups:   where [2]
```

	where	game_outcome	n	prop
	<chr>	<dbl>	<int>	<dbl>
1	away	0	29	0.569
2	away	1	22	0.431
3	home	0	17	0.298
4	home	1	40	0.702

- The dplyr function `mutate()` adds new column(s) to your data frame.

Conditional Proportions

```
1 count(buf, where, game_outcome) %>%
2   group_by(where) %>%
3   mutate(prop = n/sum(n))
```

A tibble: 4 × 4

Groups: where [2]

	where	game_outcome	n	prop
	<chr>	<dbl>	<int>	<dbl>
1	away	0	29	0.569
2	away	1	22	0.431
3	home	0	17	0.298
4	home	1	40	0.702

```
1 count(buf, where, game_outcome) %>%
2   group_by(game_outcome) %>%
3   mutate(prop = n/sum(n))
```

A tibble: 4 × 4

Groups: game_outcome [2]

	where	game_outcome	n	prop
	<chr>	<dbl>	<int>	<dbl>
1	away	0	29	0.630
2	away	1	22	0.355
3	home	0	17	0.370
4	home	1	40	0.645

How does the interpretation change based on which variable you condition on?

dplyr : go wrangling



Data Wrangling: Transformations done on the data

Why wrangle the data?

To **summarize** the data.

→ To compute the mean and standard deviation of the bike counts.

To **drop** missing values. (Need to be careful here!)

→ On our P-Set 3, we will see that **ggplot2** will often drop observations before creating a graph.

To **filter** to a particular subset of the data.

→ To subset down to just games involving Buffalo.

Data Wrangling: Transformations done on the data

Why wrangle the data?

To **arrange** the data to make it easier to display.

To fix how **R stores** a variable.

→ To **join** data frames when information about your cases is stored in multiple places!

→ To sort from the a game's best point differential to the worst.

→ To convert **game_outcomes** to be a **factor()** so that **R** would treat it like a categorical variable.

Will see examples of this next class!

dplyr for Data Wrangling

- Seven common wrangling verbs:

- `summarize()`
- `count()`
- `mutate()`
- `select()`
- `filter()`
- `arrange()`
- `---_join()`

- One action:

- `group_by()`

Return to `mutate()`

Add new variables

```
1 count(buf, where, game_outcome) %>%
2   group_by(where) %>%
3   mutate(prop = n/sum(n))
```

A tibble: 4 × 4

Groups: where [2]

	where	game_outcome	n	prop
	<chr>	<dbl>	<int>	<dbl>
1	away	0	29	0.569
2	away	1	22	0.431
3	home	0	17	0.298
4	home	1	40	0.702

Modify existing variables

```
1 class(buf$game_outcome)
```

[1] "numeric"

```
1 buf <- buf %>%
2   mutate(game_outcome = as.factor(game_outcome))
3 class(buf$game_outcome)
```

[1] "factor"

select(): Extract variables

```
1 buf %>%  
2   select(where, game_outcome)
```

```
# A tibble: 108 × 2  
  where game_outcome  
  <chr> <fct>  
1 away  1  
2 home  0  
3 away  0  
4 away  1  
5 home  1  
6 away  1  
7 away  1  
8 home  0  
9 home  0  
10 away 0  
# i 98 more rows
```

Motivation for `filter()`

```
1 count(nfl_outcomes_2018, home_team, sort = TRUE)
```

```
# A tibble: 33 × 2
```

	home_team	n
	<chr>	<int>
1	KC	64
2	BUF	57
3	SF	55
4	BAL	54
5	NO	54
6	TB	54
7	GB	53
8	HOU	53
9	LA	53
10	PHI	53

```
# i 23 more rows
```

filter(): Extract cases

```
1 buf <- filter(nfl_outcomes_2018, home_team == "BUF" | away_team == "BUF")
2 count(buf, home_team, sort = TRUE)
```

```
# A tibble: 29 × 2
```

	home_team	n
	<chr>	<int>
1	BUF	57
2	MIA	6
3	NE	6
4	NYJ	6
5	KC	5
6	TEN	3
7	BAL	2
8	HOU	2
9	ARI	1
10	CHI	1

```
# i 19 more rows
```

```
1 # Why is this useful?
2 buf <- buf %>%
3   mutate(where = if_else(home_team == "BUF", "home", "away"))
4 count(buf, where)
```

```
# A tibble: 2 × 2
```

	where	n
	<chr>	<int>
1	away	51
2	home	57

arrange(): Sort the cases

```
1 buf %>%
2   arrange(score_diff) %>%
3   select(home_team, away_team, home_score, away_score,
# A tibble: 108 × 5
  home_team away_team home_score away_score score_diff
  <chr>      <chr>      <dbl>    <dbl>    <dbl>
1 MIA        BUF          0         35      -35
2 WAS        BUF          3         37      -34
3 BUF        CHI          9         41      -32
4 NYJ        BUF         10         41      -31
5 DEN        BUF         19         48      -29
6 NE         BUF          9         38      -29
7 NYJ        BUF         17         45      -28
8 BUF        IND         15         41      -26
9 NO         BUF          6         31      -25
10 CHI       BUF         13         35      -22
# i 98 more rows
```

```
1 buf %>%
2   arrange(desc(score_diff)) %>%
3   select(home_team, away_team, home_score, away_score,
# A tibble: 108 × 5
  home_team away_team home_score away_score score_diff
  <chr>      <chr>      <dbl>    <dbl>    <dbl>
1 BAL        BUF         47          3        44
2 BUF        HOU         40          0        40
3 BUF        PIT         38          3        35
4 BUF        TEN         41          7        34
5 IND        BUF         37          5        32
6 BUF        MIA         56         26        30
7 BUF        NE          47         17        30
8 BUF        LV          38         10        28
9 BUF        MIA         48         20        28
10 TEN       BUF         42         16        26
# i 98 more rows
```

**Will see more data wrangling
next week!**

Reminders

- Code a Valentine: Feb 10th, Taylor 212